AGRITECH

AI-Powered Plant Disease Detection & Soil Classification System

Project Documentation

(College Project Report)

A web-based agricultural assistance system that uses deep-learning image embeddings to detect plant leaf diseases and classify soil types, and then provides treatment techniques and crop recommendations for practical farming decisions.

Frontend: React.js | Backend: FastAPI | AI: TensorFlow / Keras / EfficientNetB0

Core Modules: Plant Disease Detection · Soil Detection

Repository: https://github.com/crispyvicky/agritech.git

ABSTRACT

Plant disease identification and soil classification are important tasks in agriculture. Manual inspection is time-consuming and depends on expert knowledge. This project presents AgriTech, a web-based AI system that analyzes uploaded photographs of plant leaves and soil. The system uses a Convolutional Neural Network based feature extractor (EfficientNetB0 pretrained on ImageNet) to convert each image into an embedding vector. Class labels are assigned by cosine-similarity matching against class prototypes built from PlantDoc-style leaf images and soil training images. The platform also provides treatment tips for diseased leaves and crop suitability recommendations for soil types. Unrelated images are rejected with clear user guidance. The application is implemented with React.js and FastAPI and is suitable for college demonstration using a curated DEMO_TESTED_IMAGES pack.

Keywords: Plant disease detection, Soil classification, EfficientNetB0, Cosine similarity, Transfer learning, FastAPI, React.js, Agricultural decision support.

CHAPTER – 1
INTRODUCTION

1.1 Introduction

Agriculture depends on timely decisions about crop health and soil suitability. Farmers, students, and agronomy learners often rely on manual visual inspection of leaves and soil, which is slow and may give inconsistent results. Early detection of leaf disease and correct identification of soil type can reduce crop loss, guide treatment choices, and support better crop planning for a given field.

With the increasing availability of smartphones and digital cameras, image-based decision support has become practical. However, raw photographs vary in lighting, angle, and background. A useful system must not only classify an image, but also explain what the user should do next—for example which treatment technique to apply for a diseased leaf, or which crops can grow well in a given soil type.

AgriTech is proposed as an intelligent web application that simplifies this process. The user uploads a clear photograph through a browser interface. The backend runs a Machine Learning vision pipeline based on EfficientNetB0 embeddings and nearest-class prototype matching. The system returns a prediction with confidence along with practical guidance. For plant leaves, it reports the disease (or healthy) class and a short treatment tip when disease is present. For soil images, it reports the soil type and suggests suitable crops with basic care tips.

The application is developed using React.js for the interactive frontend, FastAPI for REST APIs, and TensorFlow/Keras for deep feature extraction. The presentation build focuses on two working image tools: Plant Disease Detection and Soil Detection. Verified demo images are kept in a dedicated folder so live demos remain reliable.

1.2 Problem Statement

Identifying plant diseases and soil types from photographs is difficult without domain expertise. Users often upload random photos, mixed backgrounds, or the wrong kind of image (for example a soil photo into a plant tool). A naive classifier may still force a class label with very low confidence, which confuses the user and reduces trust during evaluation or demonstration.

Traditional methods of maintaining crop notes or using general photo apps do not provide agricultural guidance such as treatment steps or crop suitability. Spreadsheet records also fail to connect a visual symptom on a leaf with an actionable recommendation. Therefore, there is a need for an intelligent and user-friendly system that can:

- Accept a leaf or soil image through a simple web UI.
- Run a clear Machine Learning algorithm (deep embedding + similarity matching).
- Reject unrelated uploads with an explicit message (plant leaf only / soil only).
- Return treatment techniques for diseases and crop suggestions for soil types.
- Support a clean live demonstration with known-good sample images.

The AgriTech system addresses these challenges by integrating modern web technologies with AI-driven image analysis into a centralized platform for agricultural image assistance.

1.3 Objective of Project

The main objective of AgriTech is to develop an intelligent and user-friendly system that helps individuals analyze plant and soil images and make informed agricultural decisions. The key objectives of the proposed system are:

1. To detect plant leaf diseases from a single uploaded leaf image using deep learning embeddings.
2. To classify common soil types (Alluvial, Black, Clay, Red) from a soil photograph.
3. To clearly document and implement the Machine Learning algorithm used (EfficientNetB0 + cosine similarity matching).
4. To reject unrelated images (soil uploaded to the plant tool, or leaf uploaded to the soil tool).
5. To display treatment tips / techniques for detected diseases.
6. To suggest suitable crops and care tips for each soil type (what can be grown).
7. To provide a clean web interface suitable for college presentation and viva demonstration.
8. To organize verified demo images in one folder for accurate live results.

1.4 Scope of the Project

The scope of AgriTech covers the development of an image-based agricultural assistance platform. The system combines web application features with AI/ML techniques to provide personalized insights and decision support. The major areas covered by the project are:

1. Plant Disease Detection

Users upload a clear leaf photo. The system returns class name, confidence, HEALTHY / DISEASED / UNCERTAIN status, top alternate predictions, and a treatment tip.

2. Soil Classification

Users upload a soil photo. The system returns soil type, confidence, notes, suitable crops, and care tips.

3. Input Validation / Wrong-Image Handling

Wrong-type images are rejected with a clear “upload plant leaf only” or “upload soil only” message instead of a forced wrong label.

4. Web Application Layer

React frontend communicates with FastAPI endpoints /predict and /predict-soil.

5. Machine Learning Pipeline

EfficientNetB0 feature extraction, prototype index building, cosine similarity ranking.

6. Guidance Knowledge Base

Disease treatments (treatments.py) and soil crop/care dictionaries.

7. Demo Assets

Verified sample images under DEMO_TESTED_IMAGES for presentation.

8. Documentation

College-style project report and a separate beginner setup/run/present guide.

CHAPTER – 2
PROBLEM IDENTIFICATION

2.1 Problem Identification

Crop health monitoring and soil assessment are essential for agriculture, but manual inspection requires experience and time. Students and farmers may not always have access to an expert when a leaf shows spots or when soil type is unclear. Mobile photos vary in lighting, angle, and background, which makes automatic recognition harder.

Managing agricultural decisions has become increasingly dependent on visual information. Individuals often need to interpret leaf symptoms and soil appearance while deciding about spraying, irrigation, and crop choice. Basic photo viewers or notebooks mainly store images; they generally provide limited analysis and do not offer personalized recommendations based on the content of the image.

The major problems identified are:

- Difficulty in recognizing leaf diseases from casual photographs.
- Confusion between healthy leaves and diseased leaves with similar colors.
- Difficulty in identifying soil type from a field photo.
- Misleading results when unrelated images (non-leaf / non-soil) are uploaded.
- Low-confidence predictions that are hard for beginners to interpret.
- Lack of immediate treatment or crop guidance after a prediction.
- Scattered test images that make demos unreliable if the wrong files are used.
- Need for a simple, documented demo path for presentations and evaluation.

2.2 Existing Problems

The existing approaches to plant/soil image assistance have the following limitations:

1. Heavy dependence on manual visual interpretation.
2. Limited automation in connecting a photo to practical advice.
3. Classifiers may force a label even when the image is unrelated.
4. Lack of clear “wrong image” messaging for beginners.
5. Limited explanation of which Machine Learning algorithm is used.
6. Limited connection between disease name and practical treatment steps.
7. Limited connection between soil type and what crops can be grown.
8. Difficulty in preparing a stable live demo without curated sample images.
9. Fragmented documentation for clone, install, and run steps.
10. Limited use of deep embeddings with similarity matching for robust demos.

2.3 Need for the Proposed System

The above problems indicate the need for an intelligent agricultural image system that can go beyond simple file upload. The proposed system is designed to analyze leaf and soil images, identify the closest known class, provide personalized recommendations (treatment / crops), reject invalid uploads, and support better decision-making. By combining React.js, FastAPI, TensorFlow/Keras EfficientNetB0 embeddings, OpenCV heuristics, and curated demo assets, the system aims to provide a centralized and efficient solution for modern educational and practical agricultural image analysis.

2.4 Related Technology Background

Computer vision for agriculture commonly uses Convolutional Neural Networks because leaves and soil textures contain local patterns (spots, veins, grain) that CNNs capture well. Transfer learning with ImageNet-pretrained networks such as EfficientNet reduces the need to train a huge model from scratch. Similarity search in embedding space is widely used in image retrieval: two images with similar meaning map to nearby vectors. AgriTech applies this idea to class prototypes so each prediction is explainable as “closest known class.”

Web integration matters for usability. A model that only runs in a notebook is hard to demonstrate to non-programmers. Exposing inference through FastAPI and presenting results in React makes the ML pipeline accessible during seminars and lab evaluations.

CHAPTER – 3
REQUIREMENTS

AgriTech requires various software tools and technologies for frontend development, backend development, and Artificial Intelligence / Machine Learning implementation.

3.1 Software Requirements

3.1.1 Operating System

Windows 10/11 (also usable on Linux or macOS).

Provides the environment required for application development and execution.

3.1.2 Frontend Technology

React.js (Create React App), Axios, React Router, Tailwind-style utility classes.

Used to develop the interactive and user-friendly interface for upload, analysis, and result display.

3.1.3 Backend Technology

FastAPI + Uvicorn.

Used to develop REST APIs and handle business logic, file upload, and JSON responses. Manages communication between the frontend and AI/ML components.

3.1.4 Programming Languages

JavaScript – frontend development. Python – backend, preprocessing, and ML inference.

Python runs TensorFlow/Keras inference, OpenCV checks, and NumPy similarity scoring.

3.1.5 Machine Learning Libraries

TensorFlow 2.x, Keras, EfficientNetB0 (ImageNet), OpenCV, NumPy, Pillow, scikit-learn (supporting stack).

Used for deep feature extraction, image preprocessing, and similarity-based class matching.

3.1.6 Datasets

PlantDoc leaf images; project soil image set (Alluvial, Black, Clay, Red).

Used to build class prototypes (average embeddings) for matching at prediction time.

3.1.7 Development Environment

Visual Studio Code / Cursor; Python virtual environment (.venv).

Local virtual environment isolates TensorFlow and API dependencies.

3.1.8 Web Browser

Google Chrome / Microsoft Edge.

Used to run the React app at http://localhost:3000.

3.1.9 Version Control

Git / GitHub.

Repository: https://github.com/crispyvicky/agritech.git

3.1.10 Package Managers

pip (Python), npm (Node.js frontend).

Used to install backend requirements and frontend node_modules.

3.2 Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent (or better).
- RAM: 8 GB minimum; 16 GB recommended for TensorFlow.
- Storage: 256 GB SSD or above (models and node_modules need space).
- Display: HD / Full HD for comfortable demo.
- Internet: Required for first-time clone and package install; local demo can run after setup.

CHAPTER – 4
DESIGN AND IMPLEMENTATION

4.1 Design

4.1.1 System Architecture

The system follows a client–server architecture. The React frontend collects an image and sends it as multipart form data to the FastAPI backend. The backend saves a temporary file, runs validation heuristics, extracts an EfficientNetB0 embedding, compares it with class prototypes, attaches guidance fields, and returns JSON. Invalid or wrong-type images return a clear error message instead of a forced guess. Temporary files are deleted after prediction.

Layer	Responsibility
Frontend (React)	Upload UI, Analyze / Predict actions, result and tip cards
API (FastAPI)	Routes /predict and /predict-soil, CORS, temp file handling
Validation	OpenCV / color heuristics to reject soil↔plant mismatches
AI (TensorFlow)	EfficientNetB0 embeddings + cosine similarity to prototypes
Guidance data	Disease treatments and soil crop / care tips
Demo assets	DEMO_TESTED_IMAGES for accurate live presentation

Table 4.1: High-level system layers

4.1.2 Data Processing Design

Uploaded images are stored temporarily under temp_uploads/. For embedding inference, images are loaded with Pillow/OpenCV, resized to 224×224, converted to RGB, preprocessed with EfficientNet preprocess_input, and passed through EfficientNetB0 with include_top=False and average pooling. The resulting feature vector is L2-normalized. Cosine similarity with each class prototype is computed using a dot product of unit vectors. The highest score becomes the predicted class; the top three scores are returned for transparency.

Confidence is derived from similarity (mapped into a 0–1 range). If the margin between the best and second-best class is small, or confidence is low, the status may be marked UNCERTAIN so the UI does not overclaim accuracy.

4.1.3 CNN / Embedding Model Design

Convolutional Neural Networks (CNNs) learn hierarchical visual features. EfficientNetB0 is a compact CNN architecture pretrained on ImageNet. In AgriTech, the classification head is removed so the network acts as a feature extractor (embedding model). This produces a fixed-length vector that represents the visual content of the uploaded photo.

Class prototypes are built by averaging embeddings of representative training images for each plant or soil class and then re-normalizing. At inference time, the query embedding is matched to the nearest prototype. This design is stable for college demos and still accepts external photos of the

same kind (clear leaf close-ups or clear soil).

4.1.4 Machine Learning Algorithm (Clear Description)

The core Machine Learning algorithm used by AgriTech is:

Deep CNN Feature Extraction (EfficientNetB0) + Nearest Prototype Matching using Cosine Similarity

Step-by-step working of the algorithm:

1. Input image I is validated (leaf vs soil heuristics where applicable).
2. I is resized to 224×224 and preprocessed for EfficientNet.
3. EfficientNetB0 (ImageNet weights, no top head) computes embedding vector v.
4. v is L2-normalized: $v \leftarrow v / \|v\|$.
5. For each class c with prototype p_c, similarity $s_c = v \cdot p_c$ (cosine similarity).
6. Predicted class = $\text{argmax}_c\ s_c$; confidence is mapped from the best similarity.
7. If similarity/margin is too low → reject or mark UNCERTAIN.
8. Attach treatment tip (plant) or crops/care tips (soil) and return JSON to the UI.

This is a form of metric / similarity-based classification on deep features, rather than only a single end-to-end softmax classifier for every demo photo. It makes the pipeline easier to explain in a viva: CNN extracts features; cosine similarity finds the closest known class; guidance tables add human-readable advice.

Item	Choice in AgriTech
Deep model	EfficientNetB0 (Keras / TensorFlow, ImageNet)
Feature type	Global average pooled embedding vector
Matching method	Cosine similarity to class prototypes
Decision rule	Nearest prototype (argmax similarity)
Uncertainty	Low similarity / small margin → UNCERTAIN or reject
Plant guidance	treatment_for(label) from treatments.py
Soil guidance	crops + care dictionary per soil class

Table 4.2: Machine Learning algorithm summary

4.2 Implementation

Implementation is split into frontend pages, FastAPI routes, and Python scripts under backend/scripts/. Models and indices are loaded lazily on first request to keep startup reasonable. CORS is configured so the React app on port 3000 can call the API on port 8000.

4.3 Module Implementation

4.3.1 Plant Disease Module

Leaf images are matched to PlantDoc-style classes (for example Tomato leaf late blight, Apple leaf, Corn rust leaf). Healthy vs diseased status is derived from the class name. When diseased, a short treatment technique is attached. Soil-like or non-leaf images are rejected with an explicit message asking for a plant leaf image only.

4.3.2 Soil Detection Module

Soil photos are matched to Alluvial, Black, Clay, or Red soil. The API returns suitable crops (what can be grown) and care tips. Leaf-heavy / green-dominant photos are rejected so the tool asks for a soil image only.

4.3.3 Frontend Modules

App.js defines routes. AgriDashboard links to the two demo tools. PlantDiseaseDetection.jsx and SoilPredictor.jsx handle file selection, API calls, loading state, error display, and result formatting including treatment / crop tips.

4.4 User Interface Design

The landing page presents AgriTech branding and a dashboard of tools. Plant Disease Detection and Soil Predictor pages keep a simple flow: Upload → Analyze/Predict → Result. Results show class, confidence, status, and tips. Error messages from the API are shown directly so wrong uploads are obvious during a demo.

4.5 System Integration

System integration combines all modules into one functional platform. The React frontend communicates with FastAPI through REST APIs. AI/ML scripts are imported from backend/scripts. Processed results are returned to the frontend and displayed through result cards, providing users with a complete plant/soil image assistance experience.

4.5.1 Frontend and Backend Integration

Axios posts FormData with the selected file to ENDPOINTS.PREDICT_DISEASE (/predict) or the soil endpoint (/predict-soil). The backend returns JSON. If error or invalid_image is present, the UI shows the message; otherwise it renders the prediction.

4.5.2 AI/ML Model Integration

load_plantdoc_predictor() and predict_soil_from_path() integrate the embedding pipeline. EfficientNetB0 weights are loaded via Keras. Prototype indices are built or loaded from disk when available. Treatments and soil_info dictionaries enrich the ML output with human-readable recommendations.

4.6 Detailed Working Flow (College Explanation)

This subsection explains the complete journey of one prediction in simple academic language.

4.6.1 Plant Disease Path

When a user uploads a leaf image on the Plant Disease Detection page, the browser creates a FormData object and posts it to POST /predict. FastAPI receives the UploadFile, writes bytes to temp_uploads/, and calls predict_disease(file_path). The predictor first checks whether the photo

looks like soil/ground using color heuristics (earth-tone ratio vs green ratio). If it looks like soil, the API returns status=invalid_image with a message asking for a plant leaf only. Otherwise, EfficientNetB0 computes embedding v. Cosine similarities against all plant class prototypes are sorted. The best class name becomes the prediction. If the best similarity is too low, the system rejects the image as “not a clear leaf.” If the margin between top-1 and top-2 is small, status becomes UNCERTAIN. Otherwise status is HEALTHY or DISEASED based on a healthy-class set. treatment_for(best_name) adds a short treatment technique string. The JSON is returned; the UI shows class, confidence percent, status color, top predictions, and the treatment tip.

4.6.2 Soil Path

When a user uploads a soil image, the frontend posts to POST /predict-soil. The soil embedder rejects leaf-like green-dominant photos. Valid soil photos are embedded and matched to Alluvial, Black, Clay, or Red prototypes. The response includes prediction, confidence, notes, crops (what can be grown), and care tips. This turns a soil label into actionable farming guidance suitable for presentation.

4.6.3 Why This Algorithm Fits a College Project

Using a pretrained CNN as a feature extractor is a standard transfer-learning pattern. Students can explain: (1) CNN layers detect edges/textures/patterns, (2) the embedding summarizes the image, (3) cosine similarity is a well-known distance/similarity measure, (4) prototypes are class centroids in feature space, (5) business rules (reject / uncertain / tips) make the system usable. This is easier to defend in a viva than a black-box label alone.

4.7 Soil Crop Guidance Summary

Soil type	Example crops that can be grown
Alluvial soil	Rice, Wheat, Corn, Sugarcane, Cotton
Black Soil	Cotton, Wheat, Jowar, Linseed, Tobacco
Clay soil	Rice, Wheat, Barley, Oats
Red soil	Millet, Groundnut, Potato, Tobacco, Pulses

Table 4.3: Soil types and suitable crops (system guidance)

4.8 Limitations (Honest Scope)

- Accuracy depends on photo quality (blur, extreme lighting, mixed backgrounds hurt results).
- The system is a decision-support demo, not a replacement for a certified agronomist.
- Soil classes are limited to four types used in the project dataset.
- External random images may be rejected or marked uncertain — this is intentional.
- Treatment tips are educational guidelines, not prescription medical/agrochemical advice.

CHAPTER – 5
CODE

Selected source fragments are shown below to illustrate the main flow (not the entire codebase). Full source lives under backend/ and frontend/.

5.1 Source Code

App.js

```
import PlantDiseaseDetection from './pages/PlantDiseaseDetection';
import SoilPredictor from './pages/SoilPredictor';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <Router>
      <Header />
      <Routes>
        <Route path="/disease-detection"
               element={<PlantDiseaseDetection />} />
        <Route path="/SoilPredictor"
               element={<SoilPredictor />} />
        <Route path="/" element={<HomeWithDashboard />} />
      </Routes>
    </Router>
  );
}
```

PlantDiseaseDetection.jsx (API call)

```
const formData = new FormData();
formData.append('file', selectedImage);
const response = await axios.post(
  ENDPOINTS.PREDICT_DISEASE, formData,
  { headers: { 'Content-Type': 'multipart/form-data' } }
);
if (data.error || data.status === 'invalid_image') {
  setError(data.error || 'Please upload a plant leaf image only.');
} else {
  setResult(data);  // class, confidence, status, treatment
}
```

main.py (API routes)

```
@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    predict_func = load_plantdoc_predictor()
    # save upload temporarily
    result = predict_func(file_path)
    os.remove(file_path)
    return result

@app.post("/predict-soil")
async def predict_soil(file: UploadFile = File(...)):
    from predict_soil_embed import predict_soil_from_path
    return predict_soil_from_path(file_path)
```

predict_plantdoc.py (algorithm core)

```
# 1) Reject soil-like photos
if _looks_like_soil_not_plant(image_path):
    return {"error": "Please upload a plant leaf image only.",
            "status": "invalid_image"}

# 2) Embed query image with EfficientNetB0
q = _embed_path(Path(image_path))   # L2-normalized vector

# 3) Cosine similarity vs each class prototype
scores = [(name, float(np.dot(q, proto))) for name, proto in index.items()]
scores.sort(key=lambda x: -x[1])
best_name, best_sim = scores[0]

# 4) Attach treatment tip
treatment = treatment_for(best_name)
return {"class": best_name, "confidence": conf,
        "status": status, "treatment": treatment}
```

EfficientNetB0 embedding load

```
_embed_model = EfficientNetB0(
    include_top=False, weights="imagenet",
    pooling="avg", input_shape=(224, 224, 3)
)
vec = model.predict(preprocess(image), verbose=0)[0]
vec = vec / (np.linalg.norm(vec) + 1e-8)
```

Soil guidance dictionary (excerpt)

```
soil_info = {
  "Alluvial soil": {
    "crops": ["Rice", "Wheat", "Corn", "Sugarcane", "Cotton"],
    "care": ["Ensure proper drainage", "Add organic matter"]
  },
  "Black Soil": {
    "crops": ["Cotton", "Wheat", "Jowar", "Linseed", "Tobacco"],
    "care": ["Improve drainage", "Add organic compost"]
  },
  "Clay soil": {
    "crops": ["Rice", "Wheat", "Barley", "Oats"],
    "care": ["Improve drainage", "Avoid working when wet"]
  },
  "Red soil": {
    "crops": ["Millet", "Groundnut", "Potato", "Pulses"],
    "care": ["Add lime if acidic", "Regular fertilization"]
  },
}
```

Treatment tip example

```
TREATMENTS = {
  "Tomato leaf late blight":
    "Remove and destroy infected plants quickly. Keep foliage dry; "
    "apply late-blight fungicide. Rotate next year.",
  "Corn rust leaf":
    "Plant resistant varieties; fungicide can help if rust appears early.",
  # ... other PlantDoc classes ...
}
```

predict_soil_embed.py (reject leaf on soil tool)

```
def predict_soil_from_path(image_path: str) -> dict:
    # Reject green/leaf-heavy photos
    if looks_like_plant_not_soil(image):
        return {
          "error": "This does not look like a soil photo. "
                   "Please upload a soil image only "
                   "(not leaves or plants).",
          "status": "invalid_image",
        }
    # Embed + match soil prototypes (Alluvial/Black/Clay/Red)
    pred, conf = nearest_soil_class(embed(image_path))
    info = soil_info[pred]
    return {
      "prediction": pred,
      "confidence": conf,
      "crops": info["crops"],
      "care": info["care"],
      "notes": info["notes"],
    }
```

start.py (server entry)

```
import os
import uvicorn

if __name__ == "__main__":
    port = int(os.environ.get("PORT", "8000"))
    uvicorn.run("main:app", host="0.0.0.0", port=port, reload=False)
```

Additional implementation files (not fully pasted here to keep the report readable): backend/scripts/predict_plantdoc.py, predict_soil_embed.py, treatments.py, frontend/src/pages/SoilPredictor.jsx, frontend/src/config/api.js, backend/main.py.

5.2 Screenshots / Demo Images of Application

Verified demo images used for accurate results are stored in AgriSync\DEMO_TESTED_IMAGES. Representative samples used during testing:



Fig 5.1 Plant Disease — Tomato late blight (diseased + treatment)

During the demo, open Plant Disease Detection, click Upload image, select this file from DEMO_TESTED_IMAGES\plant_disease, then click Analyze. Read the class name, confidence, status, and treatment tip (for diseased samples).

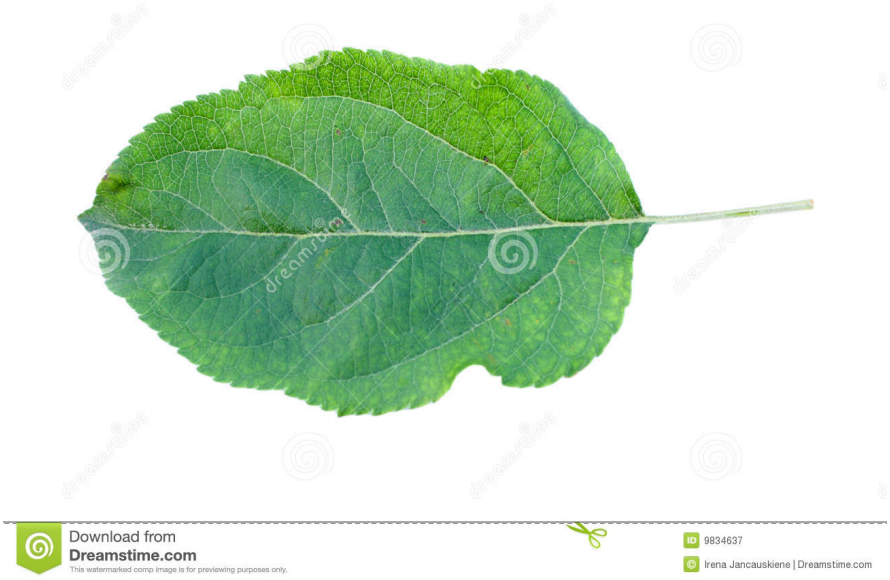


Fig 5.2 Plant Disease — Apple leaf (healthy)

During the demo, open Plant Disease Detection, click Upload image, select this file from DEMO_TESTED_IMAGES\plant_disease, then click Analyze. Read the class name, confidence, status, and treatment tip (for diseased samples).



Fig 5.3 Plant Disease — Corn rust leaf (diseased)

During the demo, open Plant Disease Detection, click Upload image, select this file from DEMO_TESTED_IMAGES\plant_disease, then click Analyze. Read the class name, confidence, status, and treatment tip (for diseased samples).



Fig 5.4 Soil — Alluvial soil



Fig 5.5 Soil — Black soil



Fig 5.6 Soil — Clay soil



Fig 5.7 Soil — Red soil

Fig 5.8 Application flow: open http://localhost:3000 → Plant Disease Detection or Soil Detection → Upload image from DEMO_TESTED_IMAGES → Analyze / Predict → read class, confidence, and tips.

CHAPTER – 6
RESULTS & CONCLUSION

6.1 RESULTS

The proposed AgriTech system was designed and implemented to provide an intelligent and user-friendly platform for plant disease detection and soil classification. The system successfully integrates React.js, FastAPI, TensorFlow/Keras EfficientNetB0 embeddings, OpenCV validation heuristics, and guidance dictionaries.

The plant module returns disease/healthy class labels with confidence and treatment tips for diseased leaves. The soil module returns soil type with crops that can be grown and care tips. Wrong-image uploads return clear guidance messages. Overall, the system achieves its objective of simplifying agricultural image analysis for demonstration and educational use.

Tool	Sample image	Expected result
Plant	01_tomato_late_blight.jpg	Tomato late blight + treatment tip
Plant	02_apple_leaf_healthy.jpg	Apple leaf (healthy)
Plant	03_corn_rust_leaf.jpg	Corn rust leaf (diseased)
Soil	01_alluvial_soil.jpg	Alluvial + crops / care tips
Soil	03_black_soil.jpg	Black Soil + crops / care tips
Soil	05_clay_soil.jpg	Clay soil + crops / care tips
Soil	06_red_soil.jpg	Red soil + crops / care tips

Table 6.1: Tested demo samples and expected outputs

6.1.1 How the System Works (End-to-End Summary)

1. User opens Plant Disease or Soil Detection in the browser.
2. User selects an image (preferably from DEMO_TESTED_IMAGES for demos).
3. Frontend posts the file to /predict or /predict-soil.
4. Backend validates image type (leaf vs soil).
5. EfficientNetB0 extracts an embedding; cosine similarity finds the nearest class.
6. Guidance (treatment or crops/care) is attached.
7. JSON is returned and shown in the UI; temp file is deleted.

6.1.2 Observation During Testing

During testing with curated demo images, plant samples such as tomato late blight, apple leaf (healthy), and corn rust leaf produced consistent class labels with treatment tips for diseased cases. Soil samples for Alluvial, Black, Clay, and Red returned the expected class along with crop lists. Cross-tool uploads (soil into plant tool and leaf into soil tool) correctly triggered rejection messages. These observations support the claim that the system is suitable for live college demonstration when DEMO_TESTED_IMAGES are used.

External images can still be used: a clear close-up of a single leaf generally works better than a wide farm scene; a clear soil surface works better than a photo dominated by grass or tools. Low-confidence / UNCERTAIN outputs should be interpreted as “try a clearer photo,” not as a forced diagnosis.

6.2 CONCLUSION

AgriTech successfully provides a centralized and intelligent platform for managing plant disease and soil image analysis. The system combines modern web technologies with a clearly defined Machine Learning algorithm—EfficientNetB0 deep embeddings with cosine-similarity prototype matching—to help users identify leaf problems and soil types and receive practical treatment and crop guidance.

Wrong-image rejection keeps the demo honest when unrelated photos are tried. The interactive UI presents results in a simple and understandable manner. Overall, the project achieves its main objective of simplifying agricultural image assistance through a user-friendly platform suitable for college-level evaluation.

The system can be further enhanced in the future by adding more plant classes, improving soil accuracy on difficult lookalikes, localized treatment guides, multilingual tips, and optional mobile camera capture for field use.

6.3 Future Enhancements

1. Expand plant disease classes and regional crop-specific tips.
2. Improve soil discrimination between visually similar clay/red samples.
3. Add optional camera capture for field use on mobile browsers.
4. Provide PDF export of a single prediction report for farmers/students.
5. Add unit/integration tests for reject rules and API contracts.
6. Package a one-click demo script for Windows presentation machines.

REFERENCES

1. React Documentation, “React – A JavaScript library for building user interfaces.” https://react.dev/ — 2026
2. FastAPI Documentation, “FastAPI – Modern Python web framework.” https://fastapi.tiangolo.com/ — 2026
3. TensorFlow Documentation, “TensorFlow – An end-to-end platform for machine learning.” https://www.tensorflow.org/ — 2026
4. Keras Documentation, “Keras API – Deep learning for humans.” https://keras.io/ — 2026
5. Tan, M. & Le, Q., “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.” ICML — 2019
6. PlantDoc Dataset – Plant disease detection images for computer vision research.
7. OpenCV Documentation, “OpenCV – Open Source Computer Vision Library.” https://docs.opencv.org/ — 2026
8. NumPy Developers, “NumPy Documentation.” https://numpy.org/doc/stable/ — 2026
9. Uvicorn Documentation, “Uvicorn – ASGI server.” https://www.uvicorn.org/ — 2026
10. Python Software Foundation, “Python Documentation.” https://docs.python.org/3/ — 2026
11. scikit-learn Developers, “Scikit-learn – Machine Learning in Python.” https://scikit-learn.org/stable/ — 2026
12. MDN Web Docs, “Using FormData objects.” https://developer.mozilla.org/ — 2026
13. GitHub, AgriSync repository. https://github.com/crispyvicky/agritech.git — 2026

APPENDIX

APPENDIX – A

System Configuration

The project was developed and tested on a computer-based system. The main software and hardware configuration used for implementation is listed below.

Hardware Configuration

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent
- RAM: 8 GB or above (16 GB recommended for TensorFlow)
- Storage: 256 GB SSD or above
- Display: Standard HD / Full HD Display
- Internet: Required for development and first package install

Software Configuration

- Operating System: Windows 10/11
- Frontend Technology: React.js
- Backend Technology: FastAPI + Uvicorn
- Programming Languages: JavaScript, Python
- Development Environment: Visual Studio Code / Cursor
- Machine Learning: TensorFlow, Keras, EfficientNetB0, OpenCV, NumPy
- Algorithm: CNN embedding + cosine similarity prototype matching
- Web Browser: Google Chrome / Microsoft Edge
- Version Control: Git / GitHub
- Package managers: pip, npm

APPENDIX – B

Dataset Information

The system uses plant leaf images (PlantDoc-style classes) and soil images for building class prototypes and evaluating predictions. Major data categories:

1. Plant leaf images (healthy and diseased classes)
2. Soil images (Alluvial, Black, Clay, Red)
3. Class prototype index (average embeddings per class)
4. Treatment tip strings per plant class
5. Crop and care tip lists per soil class
6. Verified demo copies under DEMO_TESTED_IMAGES

Demo folder path: AgriSync\DEMO_TESTED_IMAGES\plant_disease\ and AgriSync\DEMO_TESTED_IMAGES\soil\

APPENDIX – C

Major Functional Modules

1. User opens landing dashboard and selects a tool
2. Plant Disease Detection — upload leaf → analyze → class / status / treatment
3. Soil Detection — upload soil → predict → soil type / crops / care
4. Wrong-image rejection messages for plant and soil tools
5. Backend health endpoints (/health, /healthz)
6. Temporary upload cleanup after prediction
7. Demo image pack for accurate presentation results

APPENDIX – D

How to Run (Quick Reference)

```
# Backend
cd AgriSync\backend
$env:PORT="8000"
$env:PYTHONUTF8="1"
.\.venv\Scripts\python.exe start.py

# Frontend
cd AgriSync\frontend
npm start
# open http://localhost:3000

# Full beginner guide:
# docs\AgriTech_Setup_and_Demo_Guide.pdf
```

APPENDIX – E

Viva / Oral Explanation Points

Students can use the following points when explaining the project orally:

1. Problem: farmers/students need quick help reading leaf disease and soil type from photos.
2. Solution: AgriTech web app with two tools — Plant Disease Detection and Soil Detection.
3. ML algorithm: EfficientNetB0 CNN embeddings + cosine similarity to class prototypes.
4. Why not only guess labels: wrong images are rejected; uncertain cases are marked.
5. Plant output: class, confidence, HEALTHY/DISEASED, treatment technique tip.
6. Soil output: soil type, confidence, crops that can grow, care tips.
7. Stack: React frontend, FastAPI backend, TensorFlow/Keras, OpenCV, NumPy.
8. Demo reliability: use DEMO_TESTED_IMAGES folder for accurate live results.
9. Clone URL: https://github.com/crispyvicky/agritech.git
10. Setup details: see AgriTech_Setup_and_Demo_Guide.pdf for beginner steps.

APPENDIX – F

API Response Fields (for documentation)

Field	Plant / Soil meaning
class / prediction	Predicted label name

confidence	Score derived from similarity (0–1)
status	HEALTHY / DISEASED / UNCERTAIN / invalid_image
treatment	Plant: short treatment technique text
crops	Soil: list of suitable crops
care	Soil: care tip list
error	Human-readable rejection or failure message
top_predictions	Plant: alternate close classes
engine	Identifier of matching engine used

Table A.1: Important API response fields

— End of Document —